Technical University of Applied Sciences Würzburg-Schweinfurt (THWS)
Faculty of Computer Science and Business Information Systems

Master Thesis

Great Research About My Favourite Topic

submitted for the completion of degree

Master of Science

in

Artificial Intelligence

Your Name

Submitted on: November 11, 2025

Supervisor: Prof. Dr. Magda Gregorova
Second Examiner: Prof. Dr. C D

Abstract (en)

A single paragraph summarizing the most important points of your thesis. This can be more than one paragraph but should not be too long. You usually write this as the very last thing of your thesis.

Abstract (de)

We are in germany so include a translation.

Acknowledgment

Here you can thank your friends who helped you throughout the studies, your parents for supporting you generally, if you wish your supervisor or any other profs and generally everybody you value.

Contents

1 Introduction

1.1 Motivation

Scalable Vector Graphics (SVG) play an essential role in modern digital design due to their scalability, compactness, and support for semantic editing. Unlike raster images, SVGs describe visual content through structured, text-based geometry using paths, shapes, and hierarchical groupings, allowing for precise control, reusability, and easy modification.

In recent years, machine learning and deep learning models have been applied to the generation of SVGs from various input modalities, such as text prompts, raster images, and sketches. Open-source research has produced a variety of approaches, including sequence models, transformer-based architectures, and differentiable optimization methods, each proposing different strategies for representing and producing vector graphics.

Given this diversity, it becomes relevant to study how these open-source models differ in their architectures, internal representations, and the structure of the SVGs they generate. Understanding these aspects can provide a clearer picture of how SVGs are represented and produced computationally, and what factors may influence the quality and editability of the resulting graphics.

1.2 Problem Statement

While multiple open-source models exist for SVG generation, they employ heterogeneous approaches in terms of architecture, data representation, and output structure. Some models treat SVGs as sequences of drawing commands, others as hierarchical objects, and still others as continuous geometric parameters optimized through differentiable rendering.

This diversity makes it challenging to compare how different approaches handle SVG generation and how these choices influence the final output. Furthermore, the evaluation

of SVG quality often depends on various factors, such as path simplicity, structural organization, and editability, which are not always measured in the same way.

Therefore, the problem addressed in this research is to systematically describe, compare, and analyze open-source SVG generation methods, focusing on their architectures, internal representations, and output characteristics, in order to understand how these design choices relate to the quality and structure of the generated SVGs.

1.3 Research Goals

1. To explain and analyze the core properties, structures, and components of SVG graphics as a foundation for understanding their representation and generation.

2. To collect, review, and categorize existing open-source methods for SVG generation across various input modalities.

3. To investigate the types of deep learning and machine learning architectures employed in SVG generation models and analyze their design principles.

4. To examine how these architectures represent SVG data internally and how their outputs are transformed into standard SVG files.

5. To explore and evaluate existing approaches, metrics, and benchmarks for assessing the quality of generated SVGs.

6. To assess and compare the output quality of current SVG generation models based on the identified evaluation criteria.

1.4 Research Questions

1. What are the main properties and structures that define an editable and high-quality SVG?

2. What open-source methods currently exist for generating SVGs from different input modalities?

3. What types of deep learning and machine learning architectures are models used for SVG generation?

4. What kinds of structure do these models produce?

5. How can the quality and editability of generated SVG be evaluated?

6. What is the observed quality of SVG outputs across different models?

7. What makes open source model suitable for testing?

8. Which open-source SVG generation models are most suitable for testing and comparison?

9. How do the selected open-source SVG generation models perform in terms of defined criteria (eg. editability, quality and practical usage)?

2 Formatting

This chapter gives examples of some standard formatting commands and environments you may want to use. This is far from exhaustive, there are many more things you can achieve with LaTeX.

2.1 Math Equations

You may want to display math equations in three distinct styles: inline, numbered display, or non-numbered display.

A formula that appears in the running text is called an inline or in-text formula. It is produced by the **math** environment, which can be invoked with the usual `\begin{}`–`\end{}` construct or with the short form `$`...`$`. You can use any of the symbols and structures, from α to ω, available in LaTeX.

The inline style is not completely equivalent to the display style. For example, the inline equation $\lim_{x\to\infty} \frac{1}{x} = 0$ looks slightly different when set in the display style

$$\lim_{x\to\infty} \frac{1}{x} = 0 \ . \tag{2.1}$$

Here an example of an un-numbered equation

$$\int_0^{\pi/2} \cos x \, dx = \sin x \Big|_0^{\pi/2} = \sin \frac{\pi}{2} - \sin 0 = 1 \ .$$

You can use use un-numbered equations when you will not to refer to them later. The the first equation I can refer by its number as equation (2.1). Note the use of `\eqref` instead of `\ref` here and the use of `\enspace` at the end of the equation before the dot.

For a more complex equation you can use the **eqnarray** environment

$$\begin{aligned} \int_0^{\pi/2} \cos x \, dx &= \sin x \Big|_0^{\pi/2} \\ &= \sin \frac{\pi}{2} - \sin 0 = 1 \ . \end{aligned} \tag{2.2}$$

2.2 Tables

Since a table cannot be split across pages, we typically place it at the top of the page, close to its initial reference. To achieve a proper "floating" placement of tables, use the environment **table** to enclose the table's contents and caption. The contents of the table itself have to be put inside the **tabular** environment, which ensures a suitable alignment of rows and columns.

Immediately following this sentence is the point at which Table 2.1 is included in the input file; compare the placement of the table here with the table in the PDF output of this document.

Table 2.1: Frequency of Special Characters.

Non-English or Math	Frequency	Comments
Ø	1 in 1,000	Swedish names
π	1 in 5	In math
\$	4 in 5	In business
Ψ_1^2	1 in 40,000	Unexplained

2.2.1 Figures

Like tables, figures cannot be split across pages; the best placement for them is typically the top or the bottom of the page, close to their initial reference[1]. To ensure a proper "floating" placement of figures, use the environment **figure** to enclose the figure and its caption.

Figure 2.1 displays an image in the PDF format. Figure 2.2 shows a PNG image.

[1]The fourth, and last, footnote.

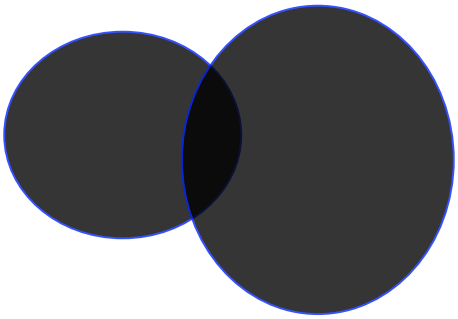

Figure 2.1: A sample circles graphic (PDF format).

Figure 2.2: A sample star graphic (PNG format).

(a) $y = x$ (b) $y = 3 \sin x$

Figure 2.3: Example of subfigre

2.2.2 Algorithms

To display algorithms in your document, employ the **algorithm** environment. Algorithms can be referenced in the same way as tables and figures (e.g., Algorithm 1).

Data: this text
Result: how to write an algorithm with LaTeX
1 initialization;
`/* this is a comment to tell you that we will now really start the code */`
2 **while** *not at end of this document* **do**
3 read the current section;
4 **if** *understand* **then**
5 go to the next section;
6 this section becomes the current section;
7 **else**
8 go back to the beginning of the current section;
9 **end**
10 **end**

Algorithm 1: How to write algorithms.

You can reference any line of your algorithm: an example of the while loop can be seen in line 2. For more details on the **algorithm** environment, see the `http://tug.ctan.org/macros/latex/contrib/algorithm2e/doc/algorithm2e.pdf` document.

3 SVG Fundamentals

List of Figures

Appendix

Great table.

Headline 1	**Headline 2**
Titel 1	Description 1
Titel 2	Description 2

Table 1: Overview in a great table.

Declaration on oath

I hereby certify that I have written my master thesis independently and have not yet submitted it for examination purposes elsewhere. All sources and aids used are listed, literal and meaningful quotations have been marked as such.

November 11, 2025, Your Name

Consent to plagiarism check

I hereby agree that my submitted work may be sent to PlagAware (https://my.plagaware.com/) in digital form for the purpose of checking for plagiarism and that it may be temporarily (max. 5 years) stored in the database maintained by PlagScan as well as personal data which are part of this work may be stored there.

Consent is voluntary. Without this consent, the plagiarism check cannot be prevented by removing all personal data and protecting the copyright requirements. Consent to the storage and use of personal data may be revoked at any time by notifying the faculty.

November 11, 2025, Your Name